

Data Types (ডেটা টাইপ)

1. Complex (কমপ্লেক্স)

কাজ: জটিল সংখ্যা (real + imaginary part) সংরক্ষণ করে

সুবিধা: গাণিতিক হিসাবে বিশেষভাবে কার্যকর

python# Complex Number Example

```
z1 = 3 + 4j
```

```
z2 = complex(2, 5) # 2+5j
```

```
print("z1 =", z1)
```

```
print("z2 =", z2)
```

```
print("z1 + z2 =", z1 + z2)
```

```
print("Real part of z1:", z1.real)
```

```
print("Imaginary part of z1:", z1.imag)
```

আউটপুট:

```
z1 = (3+4j)
```

```
z2 = (2+5j)
```

```
z1 + z2 = (5+9j)
```

```
Real part of z1: 3.0
```

```
Imaginary part of z1: 4.0
```

2. List (লিস্ট)

কাজ: একাধিক আইটেম একসাথে রাখে, পরিবর্তনযোগ্য

সুবিধা: Index দিয়ে access, append/remove করা যায়

python# List Example

```
fruits = ["আম", "কলা", "আপেল"]
```

```
numbers = [1, 2, 3, 4, 5]
```

```
mixed = [1, "Hello", 3.14, True]
```

```
print("Fruits:", fruits)
```

```
print("First fruit:", fruits[0])
```

List operations

```
fruits.append("কমলা")
```

```
print("After append:", fruits)
```

```
fruits.remove("কলা")
```

```
print("After remove:", fruits)
```

```
print("Length:", len(fruits))
```

আউটপুট:

Fruits: ['আম', 'কলা', 'আপেল']

First fruit: আম

After append: ['আম', 'কলা', 'আপেল', 'কমলা']

After remove: ['আম', 'আপেল', 'কমলা']

Length: 3

3. Tuple (টিউপল)

কাজ: একাধিক আইটেম রাখে কিন্তু পরিবর্তন করা যায় না

সুবিধা: Memory efficient, immutable, dictionary key হিসেবে ব্যবহার করা যায়

python# Tuple Example

```
coordinates = (10, 20)
```

```
colors = ("লাল", "সবুজ", "নীল")
```

```
single_item = (42,) # Comma লাগবে single item এর জন্য
```

```
print("Coordinates:", coordinates)
```

```
print("X coordinate:", coordinates[0])
```

```
print("Y coordinate:", coordinates[1])
```

```
# Tuple unpacking
```

```
x, y = coordinates
```

```
print(f"X = {x}, Y = {y}")
```

```
# Tuple methods
```

```
print("Count of 'লাল':", colors.count("লাল"))
```

```
print("Index of 'সবুজ':", colors.index("সবুজ"))
```

আউটপুট:

Coordinates: (10, 20)

X coordinate: 10

Y coordinate: 20

X = 10, Y = 20

Count of 'লাল': 1

Index of 'সবুজ': 1

4. Range (রেঞ্জ)

কাজ: একটি সংখ্যার sequence তৈরি করে

সুবিধা: Memory efficient, loop এ ব্যবহারের জন্য আদর্শ

python# Range Example

```
range1 = range(5)      # 0 to 4
```

```
range2 = range(2, 10)    # 2 to 9
range3 = range(0, 20, 3) # 0 to 19, step 3
```

```
print("Range 1:", list(range1))
print("Range 2:", list(range2))
print("Range 3:", list(range3))
```

Range in loop

```
print("Using range in loop:")
```

```
for i in range(3):
```

```
    print(f"Number: {i}")
```

আউটপুট:

Range 1: [0, 1, 2, 3, 4]

Range 2: [2, 3, 4, 5, 6, 7, 8, 9]

Range 3: [0, 3, 6, 9, 12, 15, 18]

Using range in loop:

Number: 0

Number: 1

Number: 2

5. Dictionary (ডিকশনারি)

কাজ: Key-Value pair সংরক্ষণ করে

সুবিধা: Fast lookup, flexible data storage

python# Dictionary Example

```
student = {
```

```
"নাম": "রহিম",  
"বয়স": 22,  
"শহর": "ঢাকা",  
"পেশা": "ছাত্র"  
}
```

```
print("Student info:", student)
```

```
print("নাম:", student["নাম"])
```

```
# Dictionary operations
```

```
student["গ্রেড"] = "A+"
```

```
print("After adding grade:", student)
```

```
student.update({"বয়স": 23, "বিভাগ": "CSE"})
```

```
print("After update:", student)
```

```
print("Keys:", list(student.keys()))
```

```
print("Values:", list(student.values()))
```

আউটপুট:

Student info: {'নাম': 'রহিম', 'বয়স': 22, 'শহর': 'ঢাকা', 'পেশা': 'ছাত্র'}

নাম: রহিম

After adding grade: {'নাম': 'রহিম', 'বয়স': 22, 'শহর': 'ঢাকা', 'পেশা': 'ছাত্র', 'গ্রেড': 'A+'}

After update: {'নাম': 'রহিম', 'বয়স': 23, 'শহর': 'ঢাকা', 'পেশা': 'ছাত্র', 'গ্রেড': 'A+', 'বিভাগ': 'CSE'}

Keys: ['নাম', 'বয়স', 'শহর', 'পেশা', 'গ্রেড', 'বিভাগ']

Values: ['রহিম', 23, 'ঢাকা', 'ছাত্র', 'A+', 'CSE']

6. Set (সেট)

কাজ: Unique elements সংরক্ষণ করে, duplicate থাকে না

সুবিধা: Fast membership testing, mathematical set operations

python# Set Example

```
numbers = {1, 2, 3, 4, 5}
```

```
fruits = {"আম", "কলা", "আপেল", "আম"} # duplicate "আম" removed
```

```
print("Numbers set:", numbers)
```

```
print("Fruits set:", fruits) # No duplicate "আম"
```

```
# Set operations
```

```
set1 = {1, 2, 3, 4}
```

```
set2 = {3, 4, 5, 6}
```

```
print("Union:", set1 | set2)
```

```
print("Intersection:", set1 & set2)
```

```
print("Difference:", set1 - set2)
```

```
# Adding and removing
```

```
numbers.add(6)
```

```
print("After add:", numbers)
```

```
numbers.remove(3)
```

```
print("After remove:", numbers)
```

আউটপুট:

```
Numbers set: {1, 2, 3, 4, 5}
```

Fruits set: {'কলা', 'আম', 'আপেল'}

Union: {1, 2, 3, 4, 5, 6}

Intersection: {3, 4}

Difference: {1, 2}

After add: {1, 2, 4, 5, 6}

After remove: {1, 2, 4, 5, 6}

7. Frozenset (ফ্রোজেনসেট)

কাজ: Immutable set, পরিবর্তন করা যায় না

সুবিধা: Dictionary key হিসেবে ব্যবহার করা যায়, hashable

python# Frozenset Example

```
normal_set = {1, 2, 3, 4}
```

```
frozen = frozenset([1, 2, 3, 4])
```

```
print("Normal set:", normal_set)
```

```
print("Frozen set:", frozen)
```

Frozenset as dictionary key

```
data = {
```

```
    frozenset([1, 2]): "First",
```

```
    frozenset([3, 4]): "Second"
```

```
}
```

```
print("Dictionary with frozenset keys:", data)
```

Set operations work with frozenset

```
frozen1 = frozenset([1, 2, 3])
frozen2 = frozenset([2, 3, 4])
print("Frozen union:", frozen1 | frozen2)
```

আউটপুট:

Normal set: {1, 2, 3, 4}

Frozen set: frozenset({1, 2, 3, 4})

Dictionary with frozenset keys: {frozenset({1, 2}): 'First', frozenset({3, 4}): 'Second'}

Frozen union: frozenset({1, 2, 3, 4})

Control Structures (কন্ট্রোল স্ট্রাকচার)

If-Else Statements

python# If-Else Example

```
age = 18
```

```
grade = 85
```

```
# Simple if-else
```

```
if age >= 18:
```

```
    print("আপনি প্রাপ্তবয়স্ক")
```

```
else:
```

```
    print("আপনি নাবালক")
```

```
# Multiple conditions
```

```
if grade >= 90:
```

```
    print("গ্রেড: A+")
```

```
elif grade >= 80:
```



```
print("গ্রেড: A")
```

```
elif grade >= 70:
```

```
print("গ্রেড: B")
```

```
elif grade >= 60:
```

```
print("গ্রেড: C")
```

```
else:
```

```
print("গ্রেড: F")
```

```
# Nested if
```

```
weather = "বৃষ্টি"
```

```
temperature = 25
```

```
if weather == "বৃষ্টি":
```

```
    if temperature > 20:
```

```
print("গরম বৃষ্টি - ছাতা নিন")
```

```
else:
```

```
print("ঠান্ডা বৃষ্টি - জ্যাকেট ও ছাতা নিন")
```

আউটপুট:

আপনি প্রাপ্তবয়স্ক

গ্রেড: A

গরম বৃষ্টি - ছাতা নিন

For Loop

python# For Loop Examples

List iteration

```
fruits = ["আম", "কলা", "আপেল"]
```

```
print("ফলের তালিকা:")
```

```
for fruit in fruits:
```

```
    print(f"- {fruit}")
```

```
# Range with for loop
```

```
print("\n১ থেকে ৫ পর্যন্ত:")
```

```
for i in range(1, 6):
```

```
    print(f"সংখ্যা: {i}")
```

```
# Dictionary iteration
```

```
student = {"নাম": "করিম", "বয়স": 20, "শহর": "চট্টগ্রাম"}
```

```
print("\nছাত্রের তথ্য:")
```

```
for key, value in student.items():
```

```
    print(f"{key}: {value}")
```

```
# Enumerate example
```

```
colors = ["লাল", "সবুজ", "নীল"]
```

```
print("\nরঙের তালিকা:")
```

```
for index, color in enumerate(colors):
```

```
    print(f"{index + 1}. {color}")
```

আউটপুট:

ফলের তালিকা:

- আম
- কলা
- আপেল

১ থেকে ৫ পর্যন্ত:

সংখ্যা: 1

সংখ্যা: 2

সংখ্যা: 3

সংখ্যা: 4

সংখ্যা: 5

ছাত্রের তথ্য:

নাম: করিম

বয়স: 20

শহর: চট্টগ্রাম

রঙের তালিকা:

1. লাল

2. সবুজ

3. নীল

While Loop

python# While Loop Example

count = 1

print("১ থেকে ৫ পর্যন্ত (While loop):")

```
while count <= 5:

    print(f"Count: {count}")

    count += 1

# While with break and continue

print("\n১ থেকে ১০, কিন্তু ৫ বাদ:")

num = 0

while num < 10:

    num += 1

    if num == 5:

        continue # ৫ skip করবে

    if num == 8:

        break # ৮ এ loop বন্ধ

    print(num)
```

আউটপুট:

১ থেকে ৫ পর্যন্ত (While loop):

Count: 1

Count: 2

Count: 3

Count: 4

Count: 5

১ থেকে ১০, কিন্তু ৫ বাদ:

1

2

3

4

6

7

Functions (ফাংশন)

Basic Functions

python# Basic Function

```
def greet(name):
```

```
    return f"হ্যালো, {name}!"
```

```
# Function with multiple parameters
```

```
def add_numbers(a, b):
```

```
    result = a + b
```

```
    return result
```

```
# Function with default parameter
```

```
def introduce(name, age=25):
```

```
    return f"আমার নাম {name}, বয়স {age} বছর"
```

```
# Function with multiple return values
```

```
def calculate(a, b):
```

```
    addition = a + b
```

```
    subtraction = a - b
```

```
    multiplication = a * b
```

```
    return addition, subtraction, multiplication
```

```
# Function calls
```

```
print(greet("রহিম"))
```

```
print(add_numbers(10, 15))
```

```
print(introduce("করিম"))
```

```
print(introduce("সালিম", 30))
```

```
sum_val, diff_val, prod_val = calculate(20, 5)
```

```
print(f"যোগফল: {sum_val}, বিয়োগফল: {diff_val}, গুণফল: {prod_val}")
```

আউটপুট:

হ্যালো, রহিম!

25

আমার নাম করিম, বয়স 25 বছর

আমার নাম সালিম, বয়স 30 বছর

যোগফল: 25, বিয়োগফল: 15, গুণফল: 100

Advanced Functions

python# Function with *args and **kwargs

```
def flexible_function(*args, **kwargs):
```

```
    print("Positional arguments:", args)
```

```
    print("Keyword arguments:", kwargs)
```

```
# Recursive function
```

```
def factorial(n):
```

```
    if n <= 1:
```

```
        return 1
```

```
    return n * factorial(n - 1)
```

```
# Function calls
```

```
flexible_function(1, 2, 3, name="রহিম", age=25)
```

```
print(f'৫! = {factorial(5)}')
```

আউটপুট:

Positional arguments: (1, 2, 3)

Keyword arguments: {'name': 'রহিম', 'age': 25}

৫! = 120

Lambda Functions

কাজ: ছোট anonymous functions তৈরি করে

সুবিধা: One-liner functions, functional programming এ কার্যকর

python# Basic Lambda

```
square = lambda x: x ** 2
```

```
add = lambda x, y: x + y
```

```
print("৫ এর বর্গ:", square(5))
```

```
print("৩ + ৭ =", add(3, 7))
```

```
# Lambda with built-in functions
```

```
numbers = [1, 2, 3, 4, 5]
```

```
squared = list(map(lambda x: x**2, numbers))
```

```
print("বর্গ তালিকা:", squared)
```

```
# Filter with lambda
```

```
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print("জোড় সংখ্যা:", even_numbers)
```

```
# Sort with lambda

students = [
    {"নাম": "আলি", "নম্বর": 85},
    {"নাম": "ফাতিমা", "নম্বর": 92},
    {"নাম": "করিম", "নম্বর": 78}
]

sorted_students = sorted(students, key=lambda x: x["নম্বর"],
reverse=True)

print("নম্বর অনুসারে সাজানো:")

for student in sorted_students:
    print(f'{student["নাম"]}: {student["নম্বর"]}')

```

আউটপুট:

৫ এর বর্গ: 25

৩ + ৭ = 10

বর্গ তালিকা: [1, 4, 9, 16, 25]

জোড় সংখ্যা: [2, 4]

নম্বর অনুসারে সাজানো:

ফাতিমা: 92

আলি: 85

করিম: 78

Arrays/Lists (অ্যারে/লিস্ট)

List Operations

python# List creation and operations


```
numbers = [1, 2, 3, 4, 5]

fruits = ["আম", "কলা", "আপেল"]

# Accessing elements

print("প্রথম সংখ্যা:", numbers[0])

print("শেষ ফল:", fruits[-1])

# Slicing

print("প্রথম ৩টি সংখ্যা:", numbers[:3])

print("শেষ ২টি ফল:", fruits[-2:])


# List methods

numbers.append(6)

numbers.extend([7, 8, 9])

numbers.insert(0, 0)

print("সংখ্যা তালিকা:", numbers)


# List comprehension

squares = [x**2 for x in range(1, 6)]

even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]

print("বর্গ তালিকা:", squares)

print("জোড় সংখ্যার বর্গ:", even_squares)


# 2D List (Matrix)

matrix = [

    [1, 2, 3],
```

```
[4, 5, 6],  
[7, 8, 9]  
]  
  
print("ম্যাট্রিক্স:")  
  
for row in matrix:  
  
    print(row)  
  
print("২য় সারি, ৩য় কলাম:", matrix[1][2])
```

আউটপুট:

প্রথম সংখ্যা: 1

শেষ ফল: আপেল

প্রথম ৩টি সংখ্যা: [1, 2, 3]

শেষ ২টি ফল: ['কলা', 'আপেল']

সংখ্যা তালিকা: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

বর্গ তালিকা: [1, 4, 9, 16, 25]

জোড় সংখ্যার বর্গ: [4, 16, 36, 64, 100]

ম্যাট্রিক্স:

[1, 2, 3]

[4, 5, 6]

[7, 8, 9]

২য় সারি, ৩য় কলাম: 6

Object Oriented Programming (OOP)

1. Class এবং Object

কাজ: Class হল একটি blueprint, Object হল তার instance

সুবিধা: Code reusability, modularity, encapsulation

python# Basic Class and Object

class Student:

Class variable

school_name = "ঢাকা বিশ্ববিদ্যালয়"

Constructor

def __init__(self, name, age, student_id):

self.name = name # Instance variable

self.age = age

self.student_id = student_id

self.grades = []

Instance method

def add_grade(self, grade):

self.grades.append(grade)

def get_average(self):

if self.grades:

return sum(self.grades) / len(self.grades)

return 0

def display_info(self):

avg = self.get_average()

print(f"নাম: {self.name}")

print(f"বয়স: {self.age}")

print(f"আইডি: {self.student_id}")

print(f"স্কুল: {self.school_name}")

```
print(f"গড় নম্বর: {avg:.2f}")
```

```
# Creating objects
```

```
student1 = Student("আহমেদ", 20, "CSE001")
```

```
student2 = Student("ফাতিমা", 19, "CSE002")
```

```
# Using methods
```

```
student1.add_grade(85)
```

```
student1.add_grade(90)
```

```
student1.add_grade(78)
```

```
student2.add_grade(92)
```

```
student2.add_grade(88)
```

```
print("ছাত্র ১:")
```

```
student1.display_info()
```

```
print("\nছাত্র ২:")
```

```
student2.display_info()
```

আউটপুট:

ছাত্র ১:

নাম: আহমেদ

বয়স: 20

আইডি: CSE001

স্কুল: ঢাকা বিশ্ববিদ্যালয়

গড় নম্বর: 84.33

ছাত্র ২:

নাম: ফাতিমা

বয়স: 19

আইডি: CSE002

স্কুল: ঢাকা বিশ্ববিদ্যালয়

গড় নম্বর: 90.00

2. Inheritance (উত্তরাধিকার)

কাজ: Parent class থেকে properties এবং methods পায়

সুবিধা: Code reuse, hierarchical organization

python# Parent Class

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def introduce(self):
```

```
        return f"আমি {self.name}, আমার বয়স {self.age} বছর"
```

```
    def walk(self):
```

```
        return f"{self.name} হাঁটছে"
```

Child Class 1

```
class Student(Person):
```

```
    def __init__(self, name, age, student_id):
```

```

super().__init__(name, age) # Parent constructor call

self.student_id = student_id

self.subjects = []

def study(self, subject):

    self.subjects.append(subject)

    return f"{self.name} {subject} পড়ছে"

def introduce(self): # Method overriding

    base = super().introduce()

    return f"{base} এবং আমি একজন ছাত্র (আইডি: {self.student_id})"

```

Child Class 2

```

class Teacher(Person):

    def __init__(self, name, age, subject):

        super().__init__(name, age)

        self.subject = subject

        self.students = []

    def teach(self):

        return f"{self.name} {self.subject} পড়চ্ছে"

    def add_student(self, student):

        self.students.append(student)

    def introduce(self): # Method overriding

        base = super().introduce()

        return f"{base} এবং আমি {self.subject} এর শিক্ষক"

# Creating objects

```

```
person = Person("করিম", 25)
student = Student("রহিম", 20, "CSE101")
teacher = Teacher("ড. আলি", 35, "গণিত")
print(person.introduce())
print(student.introduce())
print(teacher.introduce())
print(student.study("পাইথন"))
print(teacher.teach())
```

আউটপুট:

আমি করিম, আমার বয়স 25 বছর

আমি রহিম, আমার বয়স 20 বছর এবং আমি একজন ছাত্র (আইডি: CSE101)

আমি ড. আলি, আমার বয়স 35 বছর এবং আমি গণিত এর শিক্ষক

রহিম পাইথন পড়ছে

ড. আলি গণিত পড়াচ্ছে

3. Encapsulation (এনক্যাপসুলেশন)

কাজ: Data এবং methods কো একসাথে রাখা এবং access control

সুবিধা: Data security, controlled access

pythonclass BankAccount:

```
def __init__(self, account_holder, initial_balance=0):
    self.account_holder = account_holder
    self._balance = initial_balance # Private variable
    self.__transaction_history = []
```

```

    # Public method
def deposit(self, amount):
    if amount > 0:
        self.__balance += amount

        self.__transaction_history.append(f'জমা: {amount}')

        return f'{amount} টাকা জমা হয়েছে'
    return "ভুল পরিমাণ"


    # Public method
def withdraw(self, amount):
    if 0 < amount <= self.__balance:
        self.__balance -= amount

        self.__transaction_history.append(f'উত্তোলন: {amount}')

        return f'{amount} টাকা উত্তোলন হয়েছে'
    return "অপর্যাপ্ত ব্যালেন্স বা ভুল পরিমাণ"


    # Getter method
def get_balance(self):
    return self.__balance


    # Getter method for transaction history
def get_transaction_history(self):
    return self.__transaction_history.copy()


    # Private method

```



```

def __validate_transaction(self, amount):

    return amount > 0 and isinstance(amount, (int, float))

# Using encapsulation

account = BankAccount("মি. রহমান", 1000)

print(f'প্রাথমিক ব্যালেন্স: {account.get_balance()} টাকা')

print(account.deposit(500))

print(account.withdraw(200))

print(account.withdraw(2000)) # Should fail

print(f'বর্তমান ব্যালেন্স: {account.get_balance()} টাকা')

print("লেনদেনের ইতিহাস:", account.get_transaction_history())

# This will cause error (private variable access)

# print(account.__balance) # AttributeError

```

আউটপুট:

প্রাথমিক ব্যালেন্স: 1000 টাকা

500 টাকা জমা হয়েছে

200 টাকা উত্তোলন হয়েছে

অপর্যাপ্ত ব্যালেন্স বা ভুল পরিমাণ

বর্তমান ব্যালেন্স: 1300 টাকা

লেনদেনের ইতিহাস: ['জমা: 500', 'উত্তোলন: 200']

4. Polymorphism (পলিমরফিজম)

কাজ: Same method name কিন্তু different behavior

সুবিধা: Flexible code, same interface for different objects

python# Base class

```
class Animal:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def make_sound(self):
```

```
        pass
```

```
    def move(self):
```

```
        return f"{self.name} চলছে"
```

```
# Derived classes
```

```
class Dog(Animal):
```

```
    def make_sound(self):
```

```
        return f"{self.name} ঘেউ ঘেউ করছে"
```

```
    def move(self):
```

```
        return f"{self.name} দৌড়াচ্ছে"
```

```
class Cat(Animal):
```

```
    def make_sound(self):
```

```
        return f"{self.name} মিউ মিউ করছে"
```

```
    def move(self):
```

```
        return f"{self.name} লাফাচ্ছে"
```

```

class Bird(Animal):
    def make_sound(self):
        return f"{self.name} চিঁচিঁ করছে"

    def move(self):
        return f"{self.name} উড়ছে"

# Polymorphism in action

animals = [
    Dog("টমি"),
    Cat("বিলি"),
    Bird("পোলি")
]

print("পলিমরফিজমের উদাহরণ:")

for animal in animals:
    print(f"- {animal.make_sound()}")
    print(f"- {animal.move()}")
    print()

# Function that works with any Animal object
def animal_activity(animal):
    print(f"প্রাণী: {animal.name}")
    print(f"শব্দ: {animal.make_sound()}")

```

```
print(f"চলাফেরা: {animal.move()}")
```

```
print("ফাংশনের মাধ্যমে পলিমরফিজম:")
```

```
dog = Dog("রেক্স")
```

```
animal_activity(dog)
```

আউটপুট:

পলিমরফিজমের উদাহরণ:

- টমি ঘেউ ঘেউ করছে

- টমি দৌড়াচ্ছে

- বিলি মিউ মিউ করছে

- বিলি লাফাচ্ছে

- পোলি চিঁচিঁ করছে

- পোলি উড়ছে

ফাংশনের মাধ্যমে পলিমরফিজম:

প্রাণী: রেক্স

শব্দ: রেক্স ঘেউ ঘেউ করছে

চলাফেরা: রেক্স দৌড়াচ্ছে

5. Abstraction (অ্যাবস্ট্রাকশন)

কাজ: Complex implementation hide করে, শুধু interface দেখায়

সুবিধা: Code simplicity, focus on what rather than how

```
pythonfrom abc import ABC, abstractmethod
```

Abstract Base Class

```
class Shape(ABC):  
    def __init__(self, name):  
        self.name = name  
  
    @abstractmethod  
    def calculate_area(self):  
        pass  
  
    @abstractmethod  
    def calculate_perimeter(self):  
        pass  
  
    def display_info(self):  
        area = self.calculate_area()  
        perimeter = self.calculate_perimeter()  
        print(f"আকৃতি: {self.name}")  
        print(f"ক্ষেত্রফল: {area:.2f}")  
        print(f"পরিসীমা: {perimeter:.2f}")
```

Concrete classes

```
class Rectangle(Shape):  
    def __init__(self, length, width):  
        super().__init__("আয়তক্ষেত্র")  
  
        self.length = length  
  
        self.width = width
```

```
def calculate_area(self):  
    return self.length * self.width  
  
def calculate_perimeter(self):  
    return 2 * (self.length + self.width)
```

```
class Circle(Shape):  
    def __init__(self, radius):  
        super().__init__("বৃত্ত")  
  
        self.radius = radius  
  
        self.pi = 3.14159  
  
    def calculate_area(self):  
        return self.pi * self.radius ** 2  
  
    def calculate_perimeter(self):  
        return 2 * self.pi * self.radius  
  
class Triangle(Shape):  
    def __init__(self, side1, side2, side3):  
        super().__init__("ত্রিভুজ")  
  
        self.side1 = side1  
  
        self.side2 = side2  
  
        self.side3 = side3  
  
    def calculate_area(self):  
        # Heron's formula  
  
        s = (self.side1 + self.side2 + self.side3) / 2  
  
        area = (s * (s - self.side1) * (s - self.side2) * (s - self.side3)) ** 0.5  
  
        return area
```

```
def calculate_perimeter(self):  
    return self.side1 + self.side2 + self.side3  
  
# Using abstraction  
shapes = [  
    Rectangle(10, 5),  
    Circle(7),  
    Triangle(3, 4, 5)  
]
```

```
print("অ্যাবস্ট্রাকশনের উদাহরণ:")
```

```
for shape in shapes:
```

```
    shape.display_info()
```

```
    print("-" * 30)
```

আউটপুট:

অ্যাবস্ট্রাকশনের উদাহরণ:

আকৃতি: আয়তক্ষেত্র

ক্ষেত্রফল: 50.00

পরিসীমা: 30.00

আকৃতি: বৃত্ত

ক্ষেত্রফল: 153.94

পরিসীমা: 43.98

আকৃতি: ত্রিভুজ

ক্ষেত্রফল: 6.00

পরিসীমা: 12.00

6. Multiple Inheritance (বহুবিধ উত্তরাধিকার)

কাজ: একটি class একাধিক parent class থেকে inherit করে

সুবিধা: Multiple functionalities combine করা যায়

python# Multiple Parent Classes

class Flyable:

def fly(self):

return "উড়তে পারে"

def get_flight_speed(self):

return "দ্রুত গতিতে"

class Swimmable:

def swim(self):

return "সাঁতার কাটতে পারে"

def get_swim_depth(self):

return "গভীর পানিতে"

class Animal:

def __init__(self, name, species):

self.name = name

self.species = species

def eat(self):

return f"{self.name} খাচ্ছে"

def sleep(self):

return f"{self.name} ঘুমাচ্ছে"

Multiple Inheritance

```
class Duck(Animal, Flyable, Swimmable):
```

```
    def __init__(self, name):
```

```
        super().__init__(name, "হাঁস")
```

```
    def quack(self):
```

```
        return f"{self.name} প্যাক প্যাক করছে"
```

```
    def display_abilities(self):
```

```
        print(f"নাম: {self.name}")
```

```
        print(f"প্রজাতি: {self.species}")
```

```
        print(f"উড়া: {self.fly()}")
```

```
        print(f"সাঁতার: {self.swim()}")
```

```
        print(f"বিশেষত্ব: {self.quack()}")
```

```
class Fish(Animal, Swimmable):
```

```
    def __init__(self, name):
```

```
        super().__init__(name, "মাছ")
```

```
    def breathe_underwater(self):
```

```
        return f"{self.name} পানির নিচে শ্বাস নিচ্ছে"
```

```
class Eagle(Animal, Flyable):
```

```
    def __init__(self, name):
```

```
        super().__init__(name, "ঈগল")
```

```
    def hunt(self):
```

```

        return f"{self.name} শিকার করছে"

# Creating objects and testing

duck = Duck("ডোনাল্ড")

fish = Fish("নিমো")

eagle = Eagle("ঈগলু")

print("হাঁসের ক্ষমতা:")

duck.display_abilities()

print(f"\nমাছের ক্ষমতা:")

print(f"নাম: {fish.name}, প্রজাতি: {fish.species}")

print(fish.swim())

print(fish.breathe_underwater())

print(f"\nঈগলের ক্ষমতা:")

print(f"নাম: {eagle.name}, প্রজাতি: {eagle.species}")

print(eagle.fly())

print(eagle.hunt())


# Method Resolution Order (MRO)

print(f"\nDuck class এর MRO:")

for i, cls in enumerate(Duck.__mro__, 1):

    print(f"{i}. {cls.__name__}")

```

আউটপুট:

হাঁসের ক্ষমতা:

নাম: ডোনাল্ড

প্রজাতি: হাঁস

উড়া: উড়তে পারে

সাঁতার: সাঁতার কাটতে পারে

বিশেষত্ব: ডোনাল্ড প্যাক প্যাক করছে

মাছের ক্ষমতা:

নাম: নিমো, প্রজাতি: মাছ

সাঁতার কাটতে পারে

নিমো পানির নিচে শ্বাস নিচ্ছে

ঈগলের ক্ষমতা:

নাম: ঈগলু, প্রজাতি: ঈগল

উড়তে পারে

ঈগলু শিকার করছে

Duck class এর MRO:

1. Duck
2. Animal
3. Flyable
4. Swimmable
5. object

7. Property Decorators এবং Static Methods

```
pythonclass Temperature:
```

```
    def __init__(self, celsius=0):
```

```
        self._celsius = celsius
```

```
        # Property getter
```

```
    @property
```

```

def celsius(self):
    return self._celsius

    # Property setter

@celsius.setter
def celsius(self, value):
    if value < -273.15:

        raise ValueError("তাপমাত্রা -273.15°C এর নিচে হতে পারে না")

    self._celsius = value

    # Property getter for Fahrenheit

@property
def fahrenheit(self):
    return (self._celsius * 9/5) + 32

    # Property setter for Fahrenheit

@fahrenheit.setter
def fahrenheit(self, value):
    self.celsius = (value - 32) * 5/9

    # Static method (class এর সাথে independent)

    @staticmethod
    def is_freezing(celsius_temp):
        return celsius_temp <= 0

```

Class method

```

@classmethod

```

```

def from_fahrenheit(cls, fahrenheit):
    celsius = (fahrenheit - 32) * 5/9
    return cls(celsius)

def __str__(self):
    return f'{self._celsius}°C ({self.fahrenheit}°F)'

# Using properties and methods

temp = Temperature(25)

print(f'বর্তমান তাপমাত্রা: {temp}')

# Using property setter

temp.celsius = 30

print(f'নতুন তাপমাত্রা: {temp}')

# Using Fahrenheit property

temp.fahrenheit = 100

print(f'ফারেনহাইট সেট করার পর: {temp}')

# Using static method

print(f'০°C কি জমাট? {Temperature.is_freezing(0)}')
print(f'১০°C কি জমাট? {Temperature.is_freezing(10)}')

# Using class method

temp2 = Temperature.from_fahrenheit(68)

print(f'৬৮°F = {temp2}')

আউটপুট:

বর্তমান তাপমাত্রা: 25°C (77.0°F)

```

নতুন তাপমাত্রা: 30°C (86.0°F)

ফারেনহাইট সেট করার পর: 37.77777777777778°C (100.0°F)

০°C কি জমাট? True

১০°C কি জমাট? False

৬৮°F = 20.0°C (68.0°F)